microgpt_sft.py — An Annotated Guide

LoRA Supervised Fine-Tuning in Pure Python

@nealrichter + Amazon Kiro

June 2025

Abstract

This document walks through `microgpt_sft.py`, which implements LoRA (Low-Rank Adaptation) supervised fine-tuning on top of a pretrained microgpt model. It demonstrates the complete SFT pipeline: loading a frozen base model, attaching trainable low-rank adapters, training with response-only loss masking, and merging adapters back into the base weights. All in pure Python with zero dependencies.

Source: `https://github.com/nealrichter/research/blob/main/microgpt/microgpt_sft.py`

Contents

1 Overview

`microgpt_sft.py` extends `microgpt.py` with post-training fine-tuning:

1. **Load**: Restore pretrained weights from `model.json` (frozen).
2. **Adapt**: Attach LoRA adapters to Q and V attention projections.
3. **Train**: Optimize only adapter parameters on instruction-response pairs.
4. **Merge**: Fold adapters into base weights and save `model_sft.json`.

Parameter	Value
LoRA rank (r)	4
LoRA alpha (α)	8
Target modules	attn_wq, attn_wv
Trainable params	256 (5.7% of base)
W_down init	$\mathcal{N}(0, 0.08^2)$
W_up init	zeros

Table 1: LoRA configuration

2 Model Loading and Vocabulary Extension

The pretrained model is loaded from JSON and extended with a SEP (separator) token to delineate instructions from responses:

```
import json, math, random, sys, os
random.seed(42)

# --- Load model ---
src_file = model_file if inference_only else 'model.json'
with open(src_file) as f:
    model = json.load(f)

uchars = model['vocab']
cfg = model['config']
n_layer, n_embd, n_head = cfg['n_layer'], cfg['n_embd'], cfg['
    n_head']
head_dim = n_embd // n_head
BOS = len(uchars)
SEP = len(uchars) + 1
vocab_size = len(uchars) + 2
```

The vocabulary is extended by one token:

$$V_{\text{sft}} = V_{\text{base}} \cup \{\text{SEP}\}$$

This gives the model a boundary marker between instruction and response in the training format: $[\text{BOS}, \text{instr}_1, \ldots, \text{instr}_m, \text{SEP}, \text{resp}_1, \ldots, \text{resp}_n, \text{BOS}]$.

3 Frozen Base Model

Base weights are loaded into `Value` nodes but **never updated**—gradients flow through them during backpropagation but are zeroed without applying updates:

```
state_dict = {k: [[Value(w) for w in row] for row in mat]
              for k, mat in model['weights'].items()}

# Extend embedding tables for SEP token if needed
for key in ('wte', 'lm_head'):
    if len(state_dict[key]) < vocab_size:
        state_dict[key].append([Value(random.gauss(0, 0.08))
                                for _ in range(n_embd)])

# Extend positional embeddings for longer sequences
block_size = 32
while len(state_dict['wpe']) < block_size:
    state_dict['wpe'].append([Value(random.gauss(0, 0.08))
                              for _ in range(n_embd)])

base_params = [p for mat in state_dict.values()
               for row in mat for p in row]
```

4 LoRA Adapters

Low-Rank Adaptation [1] adds a trainable low-rank decomposition to selected weight matrices while keeping the original weights frozen.

4.1 LoRA Mathematics

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times d}$, LoRA adds:

$$h = W_0 x + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}} \cdot x$$

where:

- $W_{\text{down}} \in \mathbb{R}^{r \times d}$ projects input to low-rank space
- $W_{\text{up}} \in \mathbb{R}^{d \times r}$ projects back to full dimension
- $r = 4$ is the rank (bottleneck dimension)
- $\alpha = 8$ is the scaling factor
- Only W_{up} and W_{down} are trained (256 total parameters)

4.2 Initialization

Per the LoRA paper [1], W_{down} is randomly initialized and W_{up} is initialized to zeros, so the adapter output is zero at the start of training (no perturbation to the pretrained model):

```
lora_rank = 4
lora_alpha = 8
lora_scale = lora_alpha / lora_rank

matrix = lambda nout, nin, std=0.08: \
    [[Value(random.gauss(0, std)) for _ in range(nin)]
```

```
     for _ in range(nout)]

lora_dict = {}
for i in range(n_layer):
    lora_dict[f'layer{i}.attn_wq.down'] = matrix(lora_rank, n_embd)
    lora_dict[f'layer{i}.attn_wq.up'] = [[Value(0.0)
        for _ in range(lora_rank)] for _ in range(n_embd)]
    lora_dict[f'layer{i}.attn_wv.down'] = matrix(lora_rank, n_embd)
    lora_dict[f'layer{i}.attn_wv.up'] = [[Value(0.0)
        for _ in range(lora_rank)] for _ in range(n_embd)]

lora_params = [p for mat in lora_dict.values()
               for row in mat for p in row]
```

5 LoRA-Augmented Forward Pass

The key modification is `linear_lora`, which adds the scaled adapter output to the base linear layer:

```
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def linear_lora(x, w_base, w_down, w_up):
    """h = W_base @ x + scale * W_up @ (W_down @ x)"""
    return [b + lora_scale * l
            for b, l in zip(linear(x, w_base),
                            linear(linear(x, w_down), w_up))]
```

This is applied only to the Q and V projections in attention:

```
def gpt(token_id, pos_id, keys, values):
    x = [t + p for t, p in zip(state_dict['wte'][token_id],
                               state_dict['wpe'][pos_id])]
    x = rmsnorm(x)
    for li in range(n_layer):
        x_res = x
        x = rmsnorm(x)
        q = linear_lora(x, state_dict[f'layer{li}.attn_wq'],
                lora_dict[f'layer{li}.attn_wq.down'],
                lora_dict[f'layer{li}.attn_wq.up'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear_lora(x, state_dict[f'layer{li}.attn_wv'],
                lora_dict[f'layer{li}.attn_wv.down'],
                lora_dict[f'layer{li}.attn_wv.up'])
```

The K projection and MLP are **not** adapted—following the finding in [1] that adapting Q and V is sufficient.

6 SFT Data Format

Instruction-response pairs are loaded from `input_sft.txt` in pipe-delimited format:

```
sft_data = []
for line in open('input_sft.txt'):
    if '|' in line:
```

```
        instr, resp = line.strip().split('|', 1)
        sft_data.append((instr, resp))
```

Example data:

```
fa|Alice
mb|Benjamin
```

The 2-character codes encode gender (`f`/`m`) and starting letter.

7 Response-Only Loss Masking

The critical SFT technique: compute cross-entropy loss **only on response tokens**, masking the instruction portion [3]:

```
        instr_tok = encode(instruction)
        resp_tok = encode(response)
        tokens = [BOS] + instr_tok + [SEP] + resp_tok + [BOS]
        n = min(block_size, len(tokens) - 1)
        resp_start = len(instr_tok) + 2

        keys, values = [[] for _ in range(n_layer)],
                       [[] for _ in range(n_layer)]
        losses = []
        for pos_id in range(n):
            logits = gpt(tokens[pos_id], pos_id, keys, values)
            if pos_id >= resp_start - 1:
                probs = softmax(logits)
                losses.append(-probs[tokens[pos_id + 1]].log())

        loss = (1 / len(losses)) * sum(losses)
```

The loss function for SFT:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{|R|} \sum_{j \in R} \log P(t_j \mid t_1, \ldots, t_{j-1}; \theta)$$

where R is the set of response token positions (after SEP). Instruction tokens contribute to the forward pass (building context in the KV-cache) but not to the loss gradient.

8 Optimizer: Frozen Base + Trainable Adapters

Only LoRA parameters receive gradient updates. Base model gradients are accumulated during backward but explicitly zeroed:

```
        lr_t = lr * (1 - step / total_steps)
        for i, p in enumerate(lora_params):
            m[i] = beta1 * m[i] + (1 - beta1) * p.grad
            v_buf[i] = beta2 * v_buf[i] + (1 - beta2) * p.grad **
                2
            p.data -= lr_t * (m[i] / (1 - beta1 ** (step + 1))) \
                / ((v_buf[i] / (1 - beta2**(step+1)))**0.5 + eps)
            p.grad = 0
        for p in base_params:
            p.grad = 0
```

This is the "parameter-efficient" aspect of LoRA: we train 256 parameters instead of 4,480.

9 LoRA Merge and Export

After training, the adapter weights are merged back into the base model so inference requires no LoRA-specific code:

```
    for i in range(n_layer):
        for key in (f'layer{i}.attn_wq', f'layer{i}.attn_wv'):
            w_base = state_dict[key]
            w_down = lora_dict[f'{key}.down']
            w_up = lora_dict[f'{key}.up']
            for r in range(len(w_base)):
                for c in range(len(w_base[0])):
                    w_base[r][c].data += lora_scale * sum(
                        w_up[r][k].data * w_down[k][c].data
                        for k in range(lora_rank))
```

The merge operation:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}}$$

After merging, the model is saved as `model_sft.json` and can be loaded by `microgpt.py -i model_sft.json` without any adapter code.

10 Usage

```
# Step 1: Pretrain base model (required first)
python3 microgpt.py

# Step 2: Train LoRA SFT (loads model.json, saves model_sft.json)
python3 microgpt_sft.py

# Inference only from saved SFT model
python3 microgpt_sft.py -i
python3 microgpt_sft.py -i model_sft.json

# Use merged model with base inference script
python3 microgpt.py -i model_sft.json
```

References

[1] Hu, J.E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685

[2] Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. *QLoRA: Efficient Finetuning of Quantized LLMs*. Advances in Neural Information Processing Systems 36, 2024. https://arxiv.org/abs/2305.14314

[3] Tie, G., Zhao, Z., Song, D., et al. *A Survey on Post-training of Large Language Models*. arXiv:2503.06072v3, March 2025. https://arxiv.org/abs/2503.06072

[4] Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems 30, 2017. https://arxiv.org/abs/1706.03762

[5] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. Proceedings of ICLR, 2015. https://arxiv.org/abs/1412.6980